Set Associative Mapping

- *Set associative mapping combines direct mapping with fully associative mapping by arrangement lines of a cache into sets.*
- *Set associative mapping is a compromise that exhibits the strengths of both the direct and associative approaches while reducing their drawbacks.*
- Cache is divided into a number of sets
- Each set contains a number of lines
- The sets are persistent using a direct mapping scheme
- A given block maps to any line in a given set
- The lines within each set are treated as a small fully associative cache, where any block that can save in the set can be [illegible] line inside the set.

K-way Set Associative Mapping

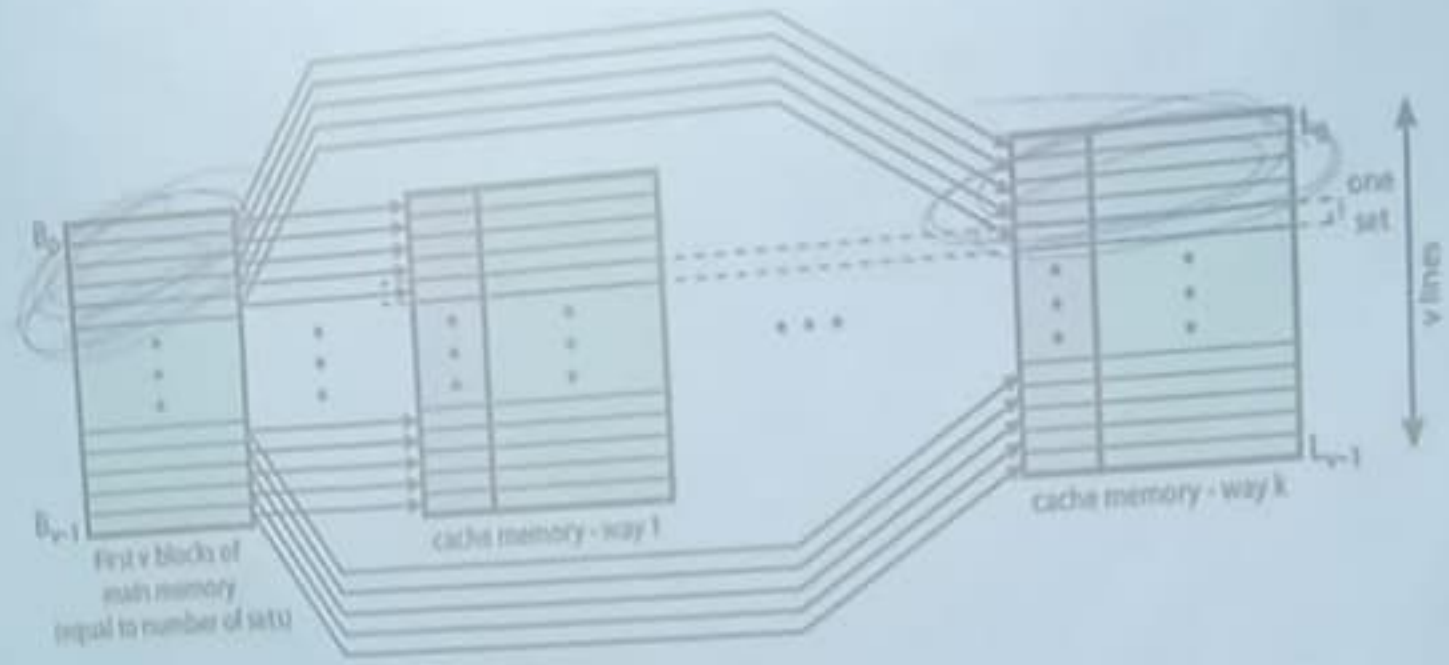
- i : cache set number
- j : main memory block number
- n : number of lines in the cache
- w : number of sets
- K : number of lines in each set

Then

1. $n = w * K$
2. $i = j \text{ modulo } w$

- Block Bj can be translated into any of the blocks in set j using this mapping.

Mapping From Main Memory to Cache: k-way Associative



K-way Set Associative Mapping

- i : cache set number
- j : main memory block number
- n : number of lines in the cache
- w : number of sets
- K : number of lines in each set

Then

1. $n = w * K$
2. $i = j$ modulo w

- Block Bj can be translated into any of the blocks in set j using this mapping.

K-way Set Associative Mapping

- Cache is divided into a number of sets
- A set-associative cache that includes k lines per set is known as a *k* way set-associative cache.
- Because the mapping approach uses the memory address only like direct mapping does.
- The number of lines included in a set should be similar to an integer power of two, for example, two, four, eight, sixteen, etc.
  - e.g., Block B can be in any line of set i
  - e.g. 2 lines per set
    - 2 way associative mapping
    - A given block can be in one of 2 lines in only one set

K-way Set Associative Mapping

- Cache is divided into a number of sets
- A set-associative cache that includes k lines per set is known as a ***k way set-associative cache***.
- Because the mapping approach uses the memory address only like direct mapping does.
- The number of lines included in a set should be similar to an integer power of two, for example, two, four, eight, sixteen, etc.
  - e.g., Block B can be in any line of set i
  - e.g. 2 lines per set
    - 2 way associative mapping
    - A given block can be in one of 2 lines in only one set

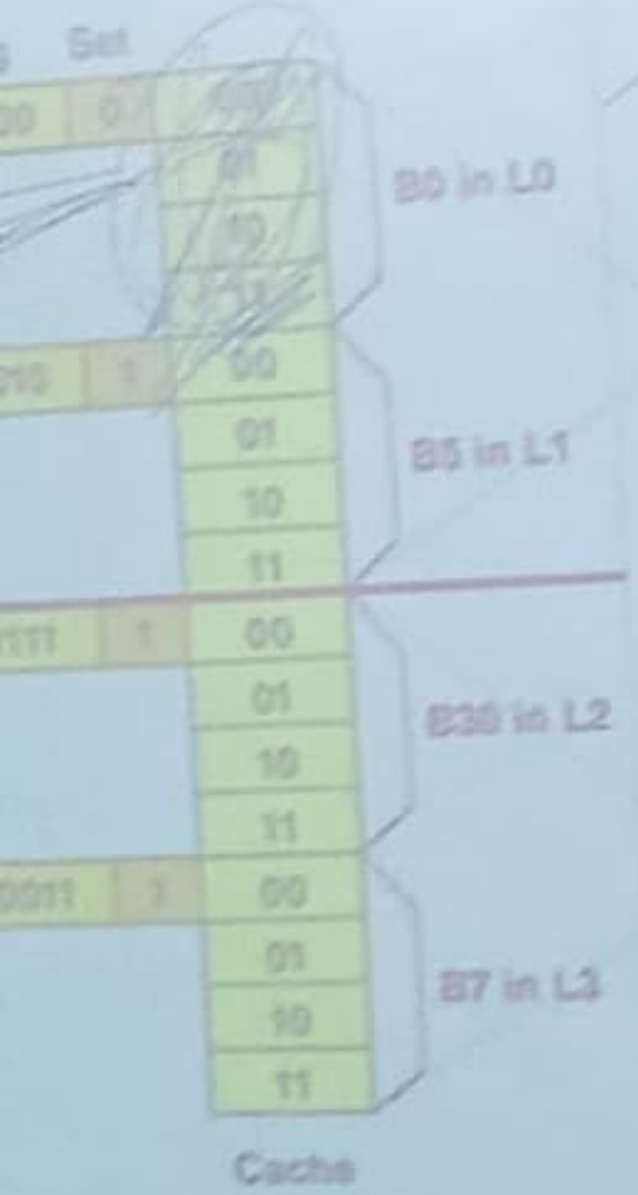
Set Associative Mapping

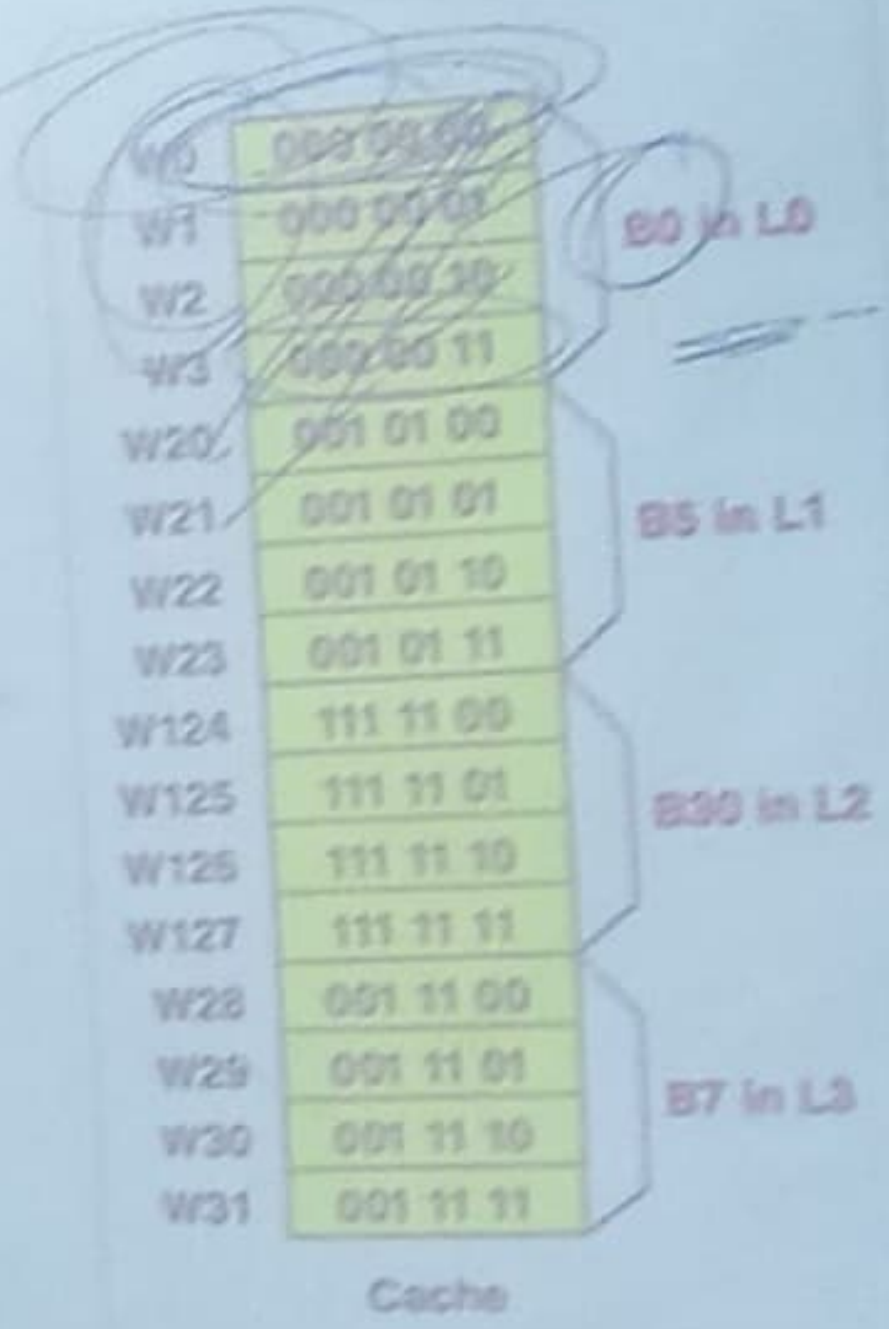
- Hybrid of Direct and Associative
  k = 1, this is basically *direct mapping*
  v = 1, this is *associative mapping*
- 2 lines per set is the most common organization.
  - Called 2 way associative mapping
  - A given block can be in one of 2 lines in only one specific set
  - Significant improvement over direct mapping

Cache set number

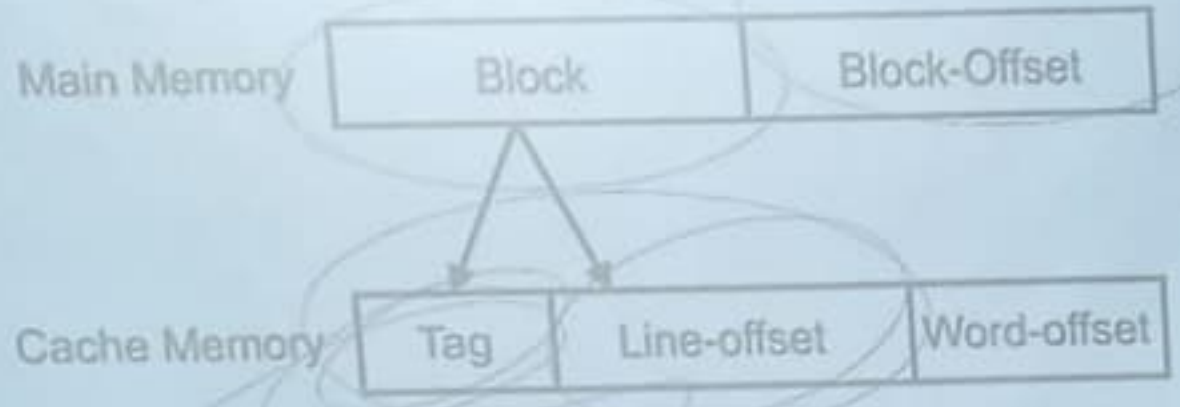
= (Main Memory Block Address) Modulo (Number of sets in Cache)

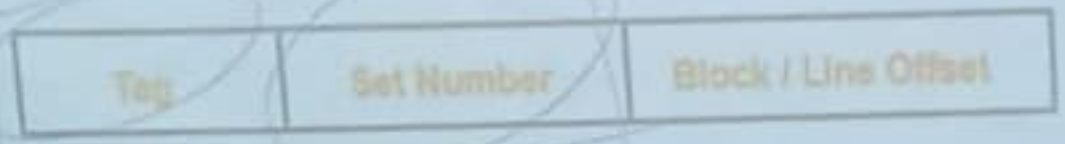
ay Set Associa





Set Associative Mapping Address Structure





Division of Physical Address in K-way Set Associative Mapping

Memory Address to Cache Mapping Example

- Suppose, we have a fully associative mapped cache of 8 KB size with block size = 128 bytes and say, the size of main memory is = 64 KB, and we have "2-way" set-associative mapping. Assume each word has 8 bits.

Solution:

Number of bits for the physical address = 16 bits (as memory size = 64 KB = $2^6 * 2^{10} = 2^{16}$)

No of cache Blocks = Cache size/block size = 8 KB / 128 Bytes = 8×1024 Bytes/128 Bytes = 2^6 cache blocks.

No of Main Memory Blocks = MM size/block size = 64 KB / 128 Bytes = 64×1024 Bytes/128 Bytes = 2^9 MM blocks.

No of sets of size 2 = No of Cache Blocks/ K = $2^6/2 = 2^5$ cache sets.(K = 2 as it is 2-way set associative mapping)

All tools Edit Export E-Sign
Generative summary

Set Associative Mapping Address Structure

Tag 9 bit	Set 13 bit	Word 2 bit

- Use set field to determine cache set to look in
- Compare tag field to see if we have a hit
- Example

	Address	Tag	Data	Set number
a.	1FF 7FFC	1FF	12345678	1FFF
b.	001 7FFC	001	11223344	1FFF

K-way Set Associative Mapping

K-way Mapping Examples (Set-Associative Mapping)

Tag	Set	Block offset

Main Memory	Cache Size	Block Size	Set Associativity	Tag Bits in TA	Tag Memory Size (bit)	Comparator (Tag)
16MB $M_0 - M_{FFF}$	16kB	32B	2-way	4	1×2^{12}	2-4
32MB $M_0 - M_{7FF}$	32kB	16B	4-way	14	14×2^{11}	4-14
64MB $M_0 - M_{3FFF}$	16MB	8B	8-way	2	2×2^{21}	8-2
16MB $M_0 - M_{FFF}$	32MB	4B	4-way	10	10×2^{19}	4-10

Word Addressable Memory

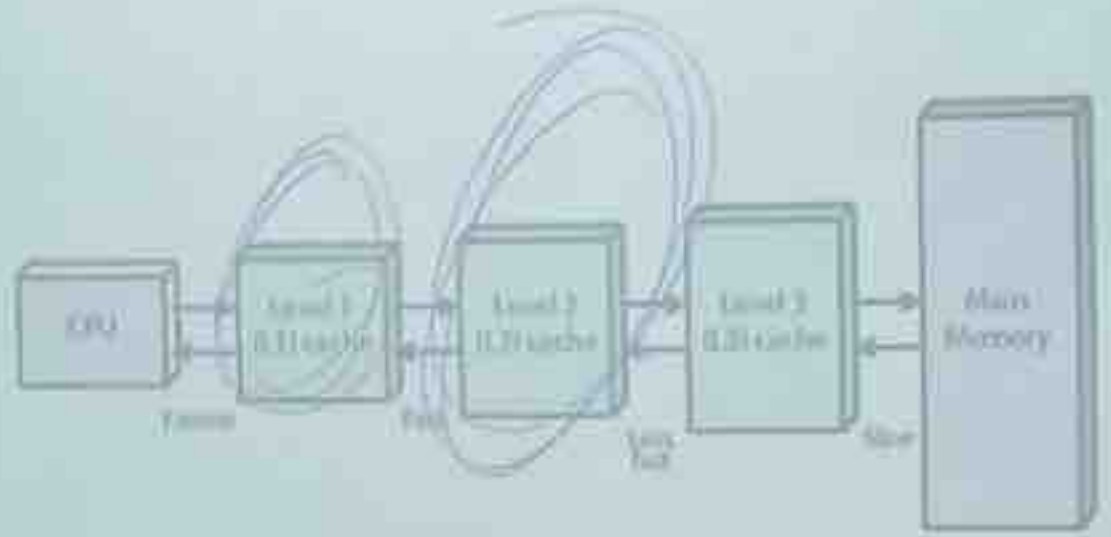
Main Memory	Cache Size	K-way	Block Size	Tag Bits in TA	Tag Memory Size (bit)	Comparator (Tag)
16MB = 2MW $M_0 - M_{1FFF}$	16MB = 4MW	2	32MB = 2MW = 1MB	4	4×2^{17}	2-4

Cache Levels

Single and Multi Level

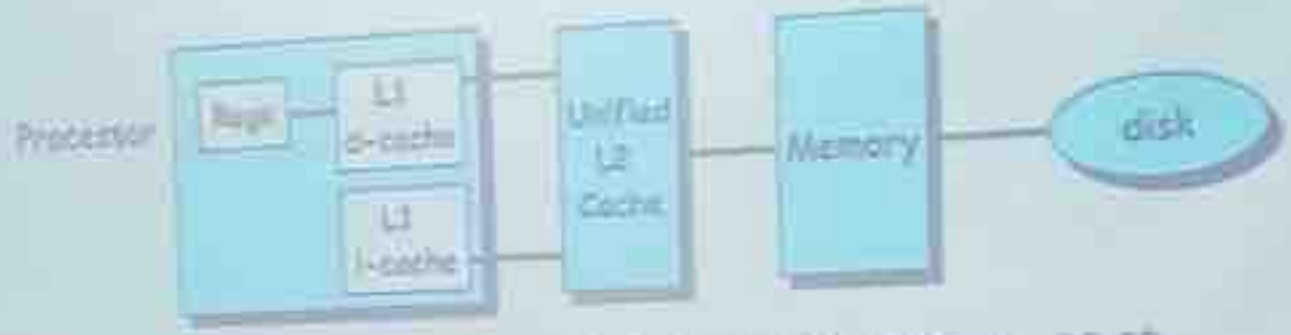
- A cache hierarchy is an arrangement of caches that cooperate with each other.
- In a cache hierarchy, the lower layers forward cache misses up to the higher layers until a cache hit is fou or the request is sent to the origin server.
- Cache hierarchies are attractive because they can off performance improvements.

Multi-Level Cache



Multi-Level Caches

Options: separate data and instruction caches, or a unified cache



size:	200 B	8-64 KB	1-4MB SRAM	128 MB DRAM	30 GB
speed:	3 ns	3 ns	6 ns	60 ns	8 ms
$/Mbyte:			$100/MB	$1.50/MB	$0.05/MB
line size:	8 B	32 B	32 B	8 KB	

larger, slower, cheaper

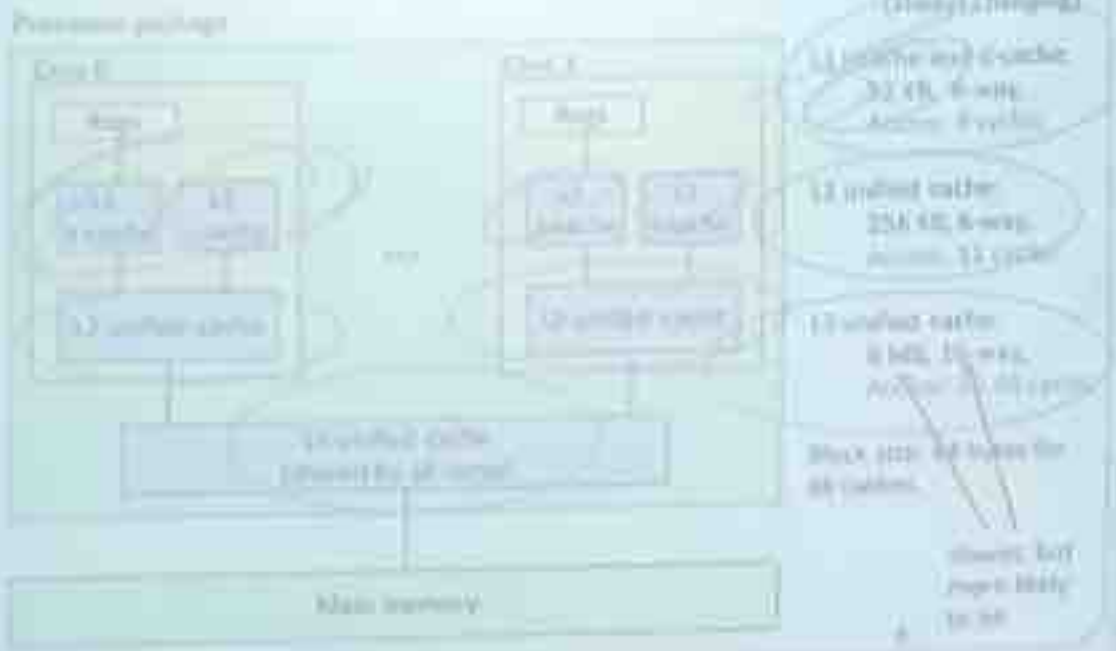
Pentium 4 Block Diagram

Types of Caches

A. Instruction and Data

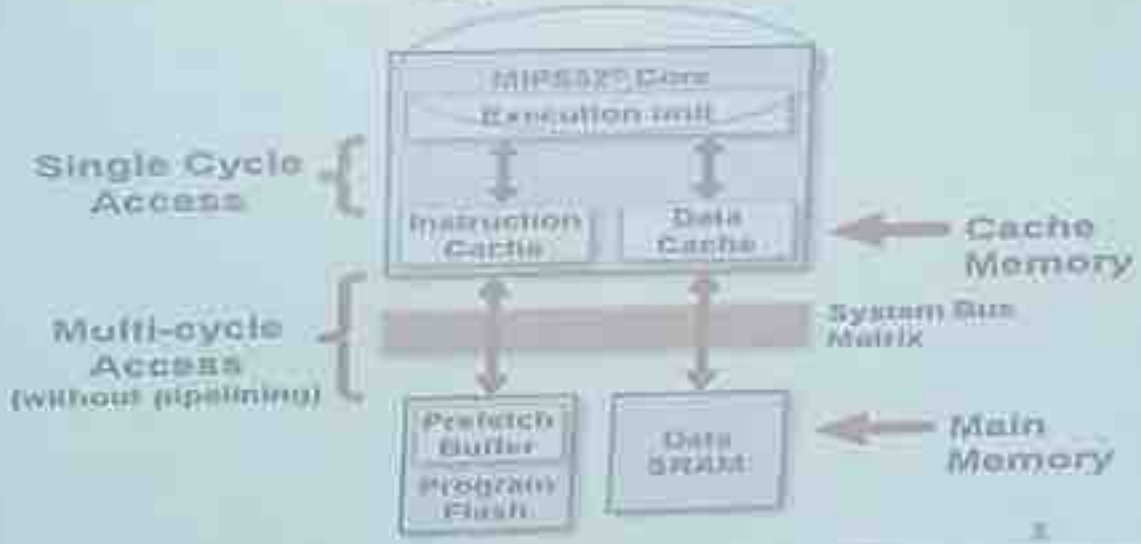
B. Unified and Split

Cache Types


Processor package
Core 0
Core 1
L1 unified cache
L2 unified cache
L3 unified cache
Main memory

Split Cache

CPU & Registers
Level 1 D Cache
Level 1 I Cache
16 KB Each
Level 2 Cache
1 or 2 MB
Main Memory


MIPS32® Core
Execution unit
Instruction Cache
Data Cache
Cache Memory
Single Cycle Access
Multi-cycle Access
(without pipelining)
System Bus Matrix
Prefetch Buffer
Program Flash
Data SRAM
Main Memory